

FIT2109 Computer Science Workshop

Week 6: Virtual Machines, Containers & Package Management

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

The problem this week solves

"It works on my machine."

The most expensive sentence in software engineering.

- Different operating systems, library versions, and system tools produce different behaviour.
- A working laptop is not the same environment as a CI server, a teammate's laptop, or a production host.
- Reproducing bugs, on-boarding new contributors, and deploying reliably all depend on solving this problem.

Shell → scripting → Git → **environments**
→ remote machines → debugging → automation

Week 6 goal

Understand the tools that make software environments reproducible: virtual machines, containers, and package managers.

Learning goals

By the end of this workshop, you should be able to:

- Explain why the same software can behave differently on different machines.
- Contrast virtual machines and containers and state when each is appropriate.
- Use core Docker commands to run, inspect, and remove containers.
- Read a Dockerfile and explain what each instruction does.
- Install packages with `apt/yum`, `pip`, and `npm`.
- Create an isolated Python environment with `venv` and pin dependencies in `requirements.txt`.
- Explain what a lockfile is and why it matters.

What tools or approaches have you heard of for managing software dependencies or running software consistently?

Concept 1: Machines differ in more than operating system

Things that differ between machines

- Operating system and version
- System libraries (libc, openssl)
- Language runtime versions (Python 3.9 vs 3.12)
- Package versions
- Environment variables and configuration

Consequences

- Tests pass locally, fail in CI.
- A library upgrade on one machine breaks another.
- A deployed app crashes in ways that cannot be reproduced locally.
- Debugging requires guessing the environment.

Concept 2: Choose the environment tool by the problem

Tool	Scope	Isolation level
Virtual machine	Full OS image	Complete (own kernel)
Container	Process + filesystem	OS-level (shared kernel)
Language package manager	Libraries only	Per-language
System package manager	OS-level tools	Machine-wide or per-user

The right tool depends on the problem

A container is not a virtual machine. A virtual environment is not a container. Each solves a different slice of the environment problem.

Concept 3: Virtual machines isolate a whole OS

How a VM works

A **hypervisor** (VMware, VirtualBox, Hyper-V) emulates hardware so a guest OS boots on the host — each guest has its own kernel, memory, and disk image.

Cost of VMs

- Full OS boot (seconds to minutes).
- Gigabytes of disk per image.
- Memory reserved per guest.

When to use a VM

- Testing on a different OS (e.g. Windows on Linux).
- Strong isolation required (own kernel, security boundary).
- Running legacy software tied to a specific OS state.

Concept 4: Containers isolate a process and filesystem

How a container works

A **container** is an isolated process with its own filesystem view, created using Linux kernel features (namespaces + cgroups).

The host kernel is shared. No second OS is booted.

Advantages over VMs

- Starts in milliseconds.
- Megabytes, not gigabytes, per image layer.
- Hundreds can run on one host.
- Identical image runs on any Linux-based host.

Limitations

- Containers share the host kernel — cannot run a Windows container on Linux natively.
- Weaker security boundary than a VM.
- On macOS/Windows, Docker runs a lightweight Linux VM internally.

Concept 5: Docker names the container workflow

Key terms

Image A read-only template. Think: a frozen filesystem + metadata.

Container A running instance of an image.

Registry A server that stores and distributes images (Docker Hub is the default).

Layer Each change to an image adds a layer; layers are cached and shared.

Analogy

Image $\approx$ class definition

Container $\approx$ instance of the class

Registry $\approx$ package repository

Workflow

Pull image $\rightarrow$ docker run $\rightarrow$ process runs
 $\rightarrow$ container exits or is stopped

A Docker **image** is best described as:

- A A running process with its own filesystem.
- B A read-only template used to create containers.
- C A virtual machine with its own kernel.
- D A list of installed packages.

Demo 1: first steps with Docker

Live on Terminal — run `make setup WEEK=6` first

Core commands for pulling, running, and inspecting containers.

```
docker pull ubuntu:22.04          # download image from Docker Hub
```

```
docker images                     # list local images
```

```
docker run ubuntu:22.04 echo "hello from a container"
```

```
docker run ubuntu:22.04 cat /etc/os-release
```

```
docker ps                         # running containers (none - already exited)
```

```
docker ps -a                     # all containers including exited
```

```
docker run --rm ubuntu:22.04 ls /usr/bin # --rm: delete container on exit
```

```
docker rm $(docker ps -aq)        # clean up stopped containers
```

Type the Docker command that lists **all** containers, including stopped ones.

Two Docker workflows: deployment vs development

Deployment-style — bake code into image

```
docker build -t myapp .  
docker run --rm myapp
```

Source is baked in. Rebuild required after every code change. Use this for CI and production.

Development-style — bind-mount live source

```
docker run --rm -it \  
-v "$PWD":/app -w /app \  
python:3.12-slim \  
sh -lc "pip install -r requirements.txt \  
&& python app.py"
```

Host directory mounted live — no rebuild needed on each edit.

Trade-off

Development-style is convenient but skips the Dockerfile. Run deployment-style before committing to confirm the image actually builds.

Concept 6: Dockerfiles describe image builds

Core instructions

FROM Base image to start from.

WORKDIR Working directory inside image.

COPY Copy files from host into image.

RUN Run a command during build.

CMD Default command on container start.

ENV Set an environment variable.

EXPOSE Document which port the app uses.

Build and run

```
# In the directory with Dockerfile:  
docker build -t myapp .  
  
docker run --rm myapp
```

Layer caching tip

Copy `requirements.txt` *before* `COPY . .` so the `pip install` layer is cached unless dependencies change.

A Dockerfile for a Python app

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
ENV PYTHONDONTWRITEBYTECODE=1    # no .pyc files during install or at runtime
RUN pip install --no-cache-dir -r requirements.txt
# --no-cache-dir: skip pip's cache => smaller image
COPY . .
CMD ["python", "app.py"]
```

Why requirements.txt before COPY . .?

Docker caches each layer. If only source changes, the cached pip install layer is reused — much faster rebuild.

Activity 1: annotate the Dockerfile

In pairs — 5 min: for each line, write what it does and *why*

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
ENV PYTHONDONTWRITEBYTECODE=1
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Discuss in pairs

- What does `--no-cache-dir` save, and why does that matter in images?
- What does `PYTHONDONTWRITEBYTECODE=1` prevent?
- Why is `COPY . .` placed *after* `pip install`?

Demo 2: build and run a Python image

Live on Terminal — workspace pre-created by `make setup WEEK=6`

```
cd seminar/demo/week6/workspace/pyapp

cat Dockerfile           # review what we are building
cat requirements.txt      # requests==2.31.0

docker build -t fit2109-pyapp . # build the image
docker images             # confirm it appeared

docker run --rm fit2109-pyapp  # run it - exits cleanly
docker run --rm -it fit2109-pyapp bash
# -it: interactive (-i) with pseudo-terminal (-t)
# inside: pip list, python --version, exit
```

After annotating the Dockerfile: what is one line you still have a question about, or one decision you would make differently?

Concept 6b: Multi-stage builds

Problem

Build tools (compilers, Maven, npm) are large but only needed at build time. Shipping them in the final image wastes space and increases attack surface.

Pattern: two FROM instructions

Stage 1 compiles. Stage 2 copies only the compiled output — no build toolchain in the final image.

```
# Stage 1: build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
COPY . .
RUN mvn -q -DskipTests package

# Stage 2: runtime only
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /src/target/app.jar .
CMD ["java", "-jar", "app.jar"]
```

Result

Maven + JDK image: ~500 MB → JRE-only image: ~200 MB. Same app, smaller footprint, reduced attack surface.

Concept 7: System packages install OS-level tools

apt — Debian, Ubuntu, WSL

```
sudo apt-get update
sudo apt-get install -y curl
sudo apt-get remove curl
apt-cache search json
```

Used in Dockerfiles: `RUN apt-get install`
...

brew — macOS

```
brew install node
brew upgrade node
brew list
```

yum / dnf — RHEL, Fedora, CentOS

```
sudo yum install curl
sudo dnf install curl      # modern alias
sudo yum remove curl
yum search json
```

Rule of thumb

apt on Ubuntu/Debian (including most Docker base images). yum/dnf on Red Hat-family systems. brew on macOS.

Demo 3: apt-get inside a container

```
docker run --rm -it ubuntu:22.04 bash
# containers run as root by default - no sudo needed inside

# inside the container:
apt-get update
apt-get install -y curl jq
curl -s https://httpbin.org/get | jq '.headers'

cat /etc/os-release          # Ubuntu 22.04 regardless of host OS
exit                         # container gone; no trace on host
```

Nothing was installed on your machine

The container had its own filesystem. `exit` discards everything.

You are on Ubuntu and need to install a system package. Which command is correct?

A `yum install curl`

B `brew install curl`

C `apt-get install curl`

D `pip install curl`

Concept 8: venv isolates; pip installs

pip — Python's package installer

```
pip install requests      # install
pip install requests==2.31.0 # pin version
pip uninstall requests
pip list                  # installed
    packages
pip show requests         # details
```

Problem: global installs conflict

Project A needs Django==3.2. Project B needs Django==4.2. A global install can only hold one version.

venv — per-project isolation

```
python3 -m venv .venv
source .venv/bin/activate # prompt: (.venv)
pip install requests
pip freeze > requirements.txt
deactivate # PATH restored; .venv/ intact
```

requirements.txt — the lockfile

```
source .venv/bin/activate
pip install -r requirements.txt
```

Demo 4: venv and requirements.txt

```
cd seminar/demo/week6/workspace/pyvenv

python3 -m venv .venv
source .venv/bin/activate      # prompt shows (.venv)

pip install requests
pip install rich==13.7.1      # pin a specific version
pip freeze > requirements.txt

deactivate                    # .venv/ still intact
cat requirements.txt          # inspect the pinned list

# Teammate recreates the same environment:
source .venv/bin/activate     # must activate first!
pip install -r requirements.txt
deactivate
```

After running `deactivate`, where are the packages installed in the `.venv` directory?

- A Still there — `deactivate` only changes your shell's `PATH`.
- B Removed — `deactivate` uninstalls the packages.
- C Moved to the global Python installation.
- D Deleted when the terminal closes.

Activity 2: fix a broken script

Solo or in pairs — 8 minutes

`broken_script.py` fails because its dependencies are missing. Your job: diagnose, fix, and lock them.

Your task

- 1 `cd seminar/demo/week6/workspace/activity2/`
- 2 `python3 -m venv .venv && source .venv/bin/activate`
- 3 Run `python broken_script.py`, read the error, `pip install` the missing package.
- 4 Repeat step 3 until the script runs without errors.
- 5 Pin the dependencies: `pip freeze > requirements.txt`, then deactivate.

Hint

`ModuleNotFoundError: No module named 'X' names the package to install.`

What is one dependency management mistake you have made (or seen) in a project, and how would you fix it now?

Concept 9: npm records Node dependencies

Key commands

```
npm install express      # runtime
                        dependency
npm install --save-dev jest # dev-only
npm list                 # installed
                        packages
```

package.json — the manifest

Records: dependencies, devDependencies, and scripts (command shortcuts). Created with `npm init`.

package-lock.json

Auto-generated. Pins *exact* versions of every package, including transitive ones. Commit this file.

node_modules/

Never commit — can be hundreds of MB. Add to `.gitignore` and regenerate with `npm install`.

Demo 5: npm init and npm install

Live on Terminal

Create a new Node project and install a package.

```
cd seminar/demo/week6/workspace/nodeapp

node --version && npm --version  # confirm installed
cat package.json                 # inspect the manifest
npm install                      # install listed dependencies
ls node_modules/ | head          # many files
cat package-lock.json | head -20 # pinned exact versions
node index.js                   # run the app

rm -rf node_modules && npm install # simulate a fresh checkout
```

A colleague clones your Node repo and runs `npm install`. Which file ensures they get the exact same package versions you have?

- A `package.json` (the manifest with version ranges)
- B `package-lock.json` (the lockfile with exact versions)
- C `node_modules/` (the installed packages directory)
- D `.npmrc` (the npm configuration file)

Concept 10: Lockfiles make versions repeatable

The pattern across ecosystems

Ecosystem	Manifest	Lockfile
Python	requirements.txt	requirements.txt*
Node.js	package.json	package-lock.json
Ruby	Gemfile	Gemfile.lock
Rust	Cargo.toml	Cargo.lock

* Python uses one file for both roles. Run `pip freeze > requirements.txt` to produce a pinned lockfile version.

Manifest vs lockfile

- **Manifest:** declares what you want, often with version ranges (≥ 2.0).
- **Lockfile:** pins exactly what was installed. Fully deterministic.

Rules

- Always commit the lockfile.
- Never hand-edit a lockfile.
- Add `node_modules/` and `.venv/` to `.gitignore`.

Rank these from **broadest** to **narrowest** isolation scope (drag to reorder):

- Virtual machine — own kernel, own OS
- Docker container — shared kernel, own filesystem
- Python venv — per-project library isolation
- Global pip install — machine-wide, no isolation

What should feel different after today?

You should now be able to

- Pull and run a Docker image, and explain what happened.
- Read a Dockerfile and explain each instruction.
- Install system packages with `apt-get` inside a container.
- Create a Python `venv`, install packages, and pin them.
- Initialise a Node project and understand `package.json`.
- Explain the difference between a manifest and a lockfile.

The idea to keep

Reproducibility = declared dependencies + locked versions + isolated runtime

Containers lock the OS layer. Lockfiles lock the library layer. Together they eliminate “works on my machine.”

Check your understanding (answer individually)

- ❶ What is the difference between a Docker image and a Docker container?
- ❷ A Dockerfile has `COPY requirements.txt .` before `COPY . .`. Why?
- ❸ You activate a `venv` and run `pip install flask`. A teammate clones the repo fresh. What must they do to get the same Flask version?
- ❹ What is the difference between `docker rm` and `docker rmi`?
- ❺ *Reflection:* What is one environment-related problem from a past project that this week's tools could have prevented?

Name one concept from today's workshop that clicked for you.

Coming up

- **Applied class:** you will containerise a small application end-to-end.
- **Week 8:** Version control with Git — branches, merges, and conflicts.

Resources

- `seminar/guides/docker_install.md` — installation instructions you should have completed before today.
- `docs.docker.com/get-started` — official Docker tutorial.
- `python3 -m venv -help` and `npm help` — built-in docs.

Questions?